

APOSTILA COMPLETA DE SEABORN

Visualização de dados com Python

Instalação • Datasets • Temas e paletas
Relacional, distribuição, categórico, matriz e regressão
FacetGrid, PairGrid, JointGrid e figure-level plots
Estilização, exportação e boas práticas

Do básico ao avançado — com código pronto para rodar

Sumário

1. O que é o Seaborn
2. Instalação e importação
3. Datasets integrados
4. Temas e paletas de cores
5. Gráficos relacionais — scatterplot e lineplot
6. Distribuições — histplot, kdeplot, ecdfplot, rugplot
7. Gráficos categóricos — box, violin, bar, count, strip, swarm
8. Matrizes — heatmap e clustermap
9. Regressão — regplot e lmlplot
10. Grades: FacetGrid, PairGrid e JointGrid
11. Figure-level vs axes-level
12. Customização, títulos e exportação
13. Integração com Matplotlib e Pandas
14. Boas práticas de visualização
15. Referência rápida (cheatsheet)

1. O que é o Seaborn

O **Seaborn** é uma biblioteca de visualização de dados em Python construída sobre o **Matplotlib** e altamente integrada ao **Pandas**. Ele oferece uma API de alto nível para criar gráficos estatísticos bonitos e informativos com poucas linhas de código.

Principais vantagens:

- Sintaxe declarativa: você diz o *que* plotar, não *como* desenhar.
- Suporte nativo a DataFrames — basta passar `data=` e nomes de colunas.
- Temas e paletas profissionais por padrão.
- Funções para facetamento automático (dividir dados em subplots).
- Estatística embutida: barras de erro, regressão, KDE, agregações.

2. Instalação e importação

```
# Instalação (terminal)
python -m pip install seaborn
```

```
# Em um notebook Jupyter
!pip install seaborn
```

Importação padrão usada em toda a apostila:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

```
sns.set_theme(style="whitegrid", palette="deep") # tema global
```

A chamada **sns.set_theme()** aplica o estilo do Seaborn a *todos* os gráficos criados na sessão, inclusive os do Matplotlib puro.

3. Datasets integrados

O Seaborn traz vários datasets prontos para prática, baixados sob demanda:

```
tips      = sns.load_dataset("tips")      # gorjetas em restaurante
iris      = sns.load_dataset("iris")      # medidas de pétalas
penguins  = sns.load_dataset("penguins")  # pinguins de Palmer
flights   = sns.load_dataset("flights")   # passageiros por mês/ano
titanic   = sns.load_dataset("titanic")   # passageiros do Titanic

print(sns.get_dataset_names())            # lista completa
print(tips.head())
```

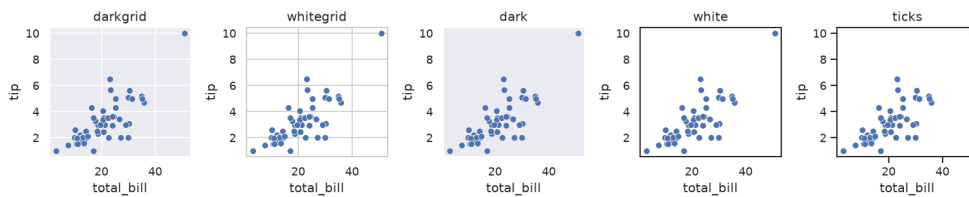
Os exemplos desta apostila usam **tips**, **penguins**, **iris** e **flights**.

4. Temas e paletas de cores

Um dos motivos da popularidade do Seaborn é a estética. Existem **5 estilos** base: *darkgrid*, *whitegrid*, *dark*, *white*, *ticks*. Cada um serve a um propósito — grids ajudam leitura de valores; estilos sem grid são melhores para apresentações.

```
sns.set_style("whitegrid")          # aplica globalmente
with sns.axes_style("dark"):        # apenas dentro do bloco
    sns.scatterplot(data=tips, x="total_bill", y="tip")

sns.set_context("talk")             # tamanho: paper|notebook|talk|poster
sns.despine()                       # remove bordas superiores/direitas
```



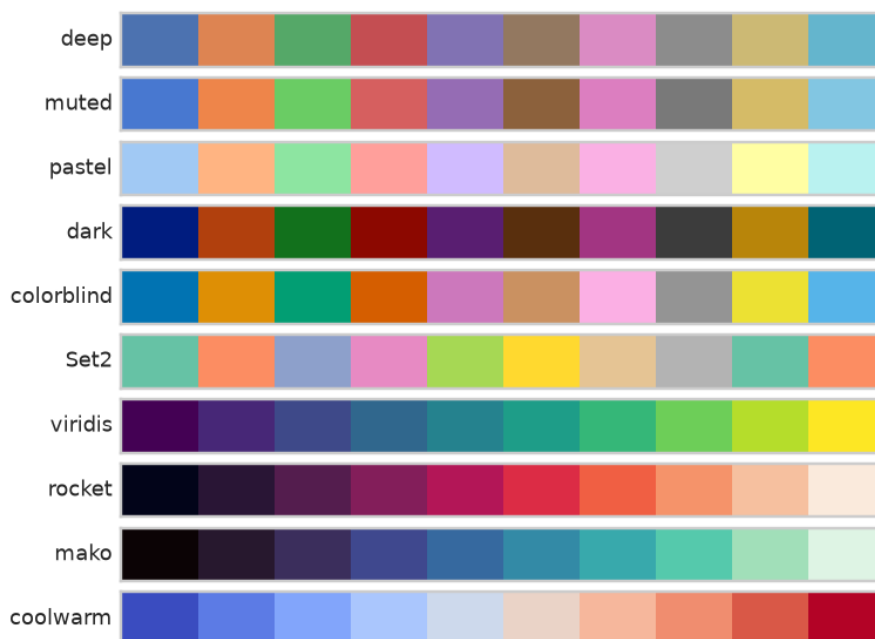
Comparação dos 5 estilos: darkgrid, whitegrid, dark, white, ticks.

Paletas

Paletas **qualitativas** (deep, muted, pastel, dark, colorblind, Set2) são ideais para categorias. Paletas **sequenciais** (viridis, mako, rocket) para variáveis contínuas. Paletas **divergentes** (coolwarm, vlag, icefire) para dados com centro (ex.: correlações).

```
sns.color_palette("Set2")          # retorna a paleta
sns.palplot(sns.color_palette("mako", 8))
sns.set_palette("colorblind")      # define globalmente
```

Paletas de cores do Seaborn



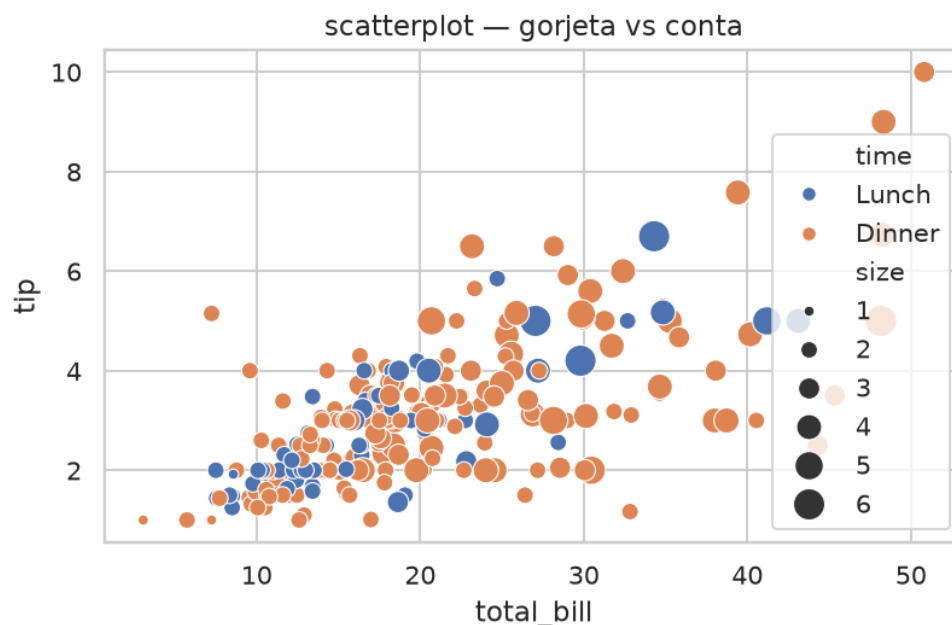
Amostras de paletas nativas do Seaborn.

5. Gráficos relacionais

Mostram a relação entre duas variáveis numéricas. As duas funções principais são **scatterplot** (pontos) e **lineplot** (linhas com estatística).

5.1 scatterplot

```
sns.scatterplot(
    data=tips, x="total_bill", y="tip",
    hue="time",          # cor por categoria
    size="size",         # tamanho por variável
    sizes=(20, 180),    # faixa de tamanhos
    style="smoker",      # forma do marcador
)
```

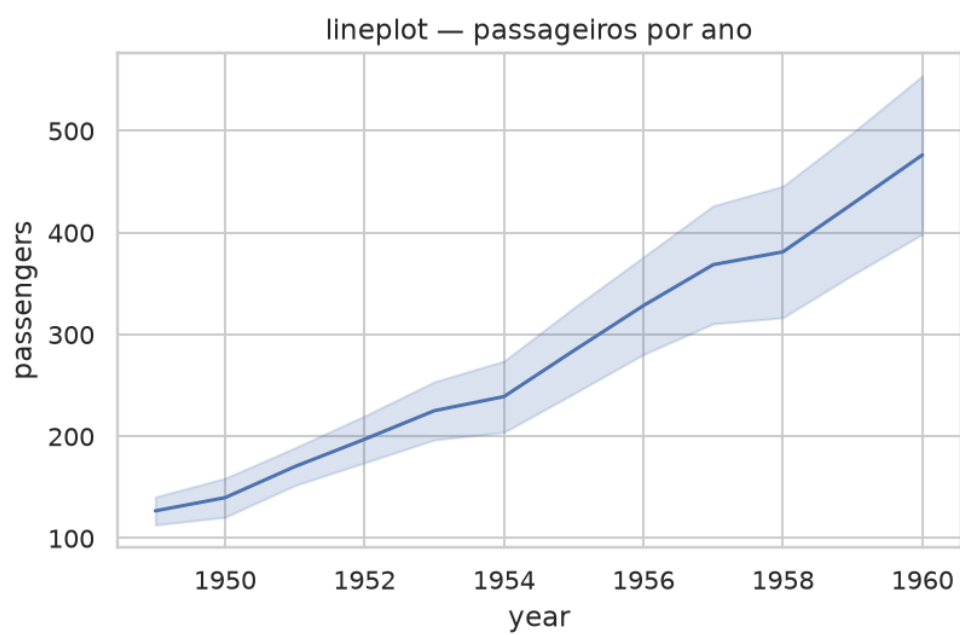


scatterplot com hue, size e categorias.

5.2 lineplot

O **lineplot** agrupa por x e calcula automaticamente média e intervalo de confiança (ou desvio padrão) do y. Ótimo para séries temporais.

```
sns.lineplot(data=flights, x="year", y="passengers", errorbar="sd")
# errorbar: 'ci', 'sd', 'se', ('pi', 95), None
```



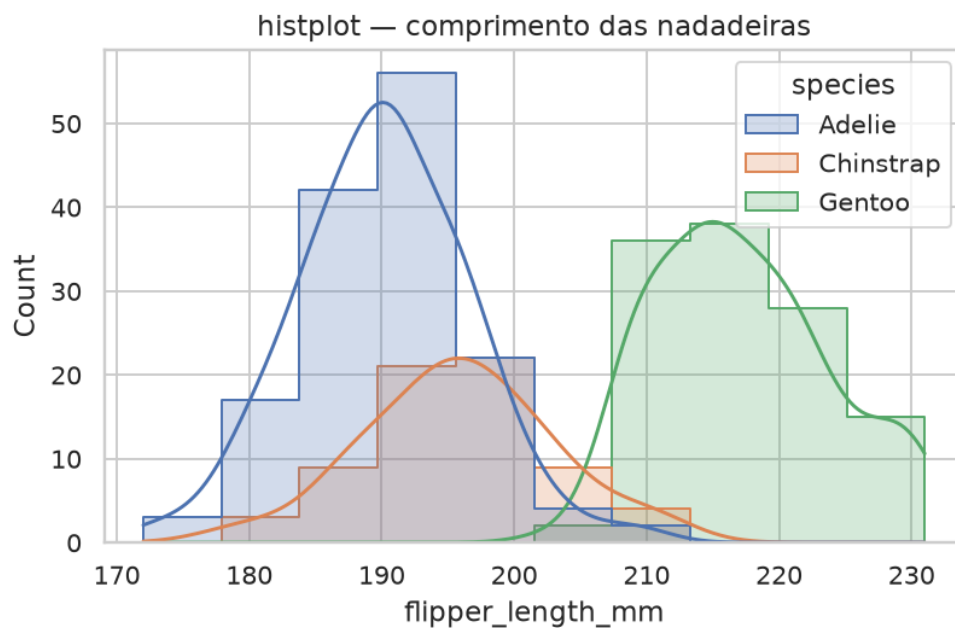
lineplot com faixa de dispersão (sd).

6. Gráficos de distribuição

Servem para entender **como** os dados estão distribuídos: forma, dispersão, caudas, assimetria, presença de multimodalidade.

6.1 histplot

```
sns.histplot(
    data=penguins, x="flipper_length_mm",
    hue="species", kde=True,
    element="step",      # step | bars | poly
    stat="count",        # count | frequency | density | probability
    bins=25,
)
```

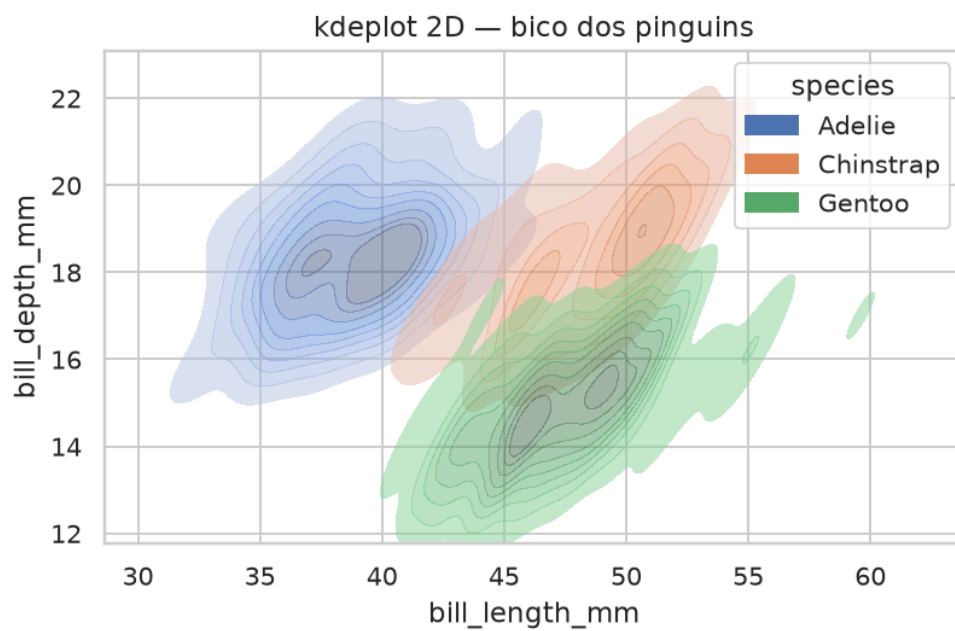


histplot com KDE por espécie.

6.2 kdeplot

O **kdeplot** estima a densidade de probabilidade por kernel. Aceita 1 ou 2 variáveis (nesse caso vira um mapa de contorno).

```
sns.kdeplot(data=penguins, x="bill_length_mm", y="bill_depth_mm",
            hue="species", fill=True, alpha=0.5, thresh=0.05)
```



kdeplot 2D com preenchimento por espécie.

6.3 Outros de distribuição

```
sns.ecdfplot(data=penguins, x="body_mass_g", hue="species") # ECDF
sns.rugplot(data=tips, x="total_bill") # "tapete" de marcas
sns.displot(data=tips, x="total_bill", kind="hist") # figure-level
```


7. Gráficos categóricos

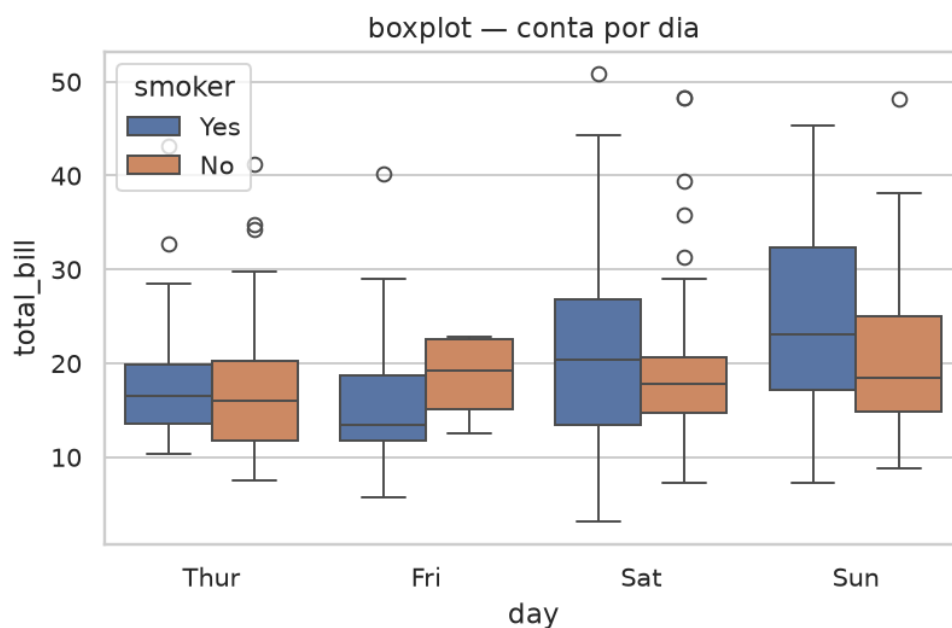
Quando o eixo x (ou y) é uma variável categórica. Cada função enfatiza um aspecto:

- **boxplot / violinplot**: distribuição resumida.
- **barplot**: valor agregado (média por padrão).
- **countplot**: contagem de ocorrências.
- **stripplot / swarmplot**: pontos individuais.
- **pointplot**: média com intervalo de confiança.

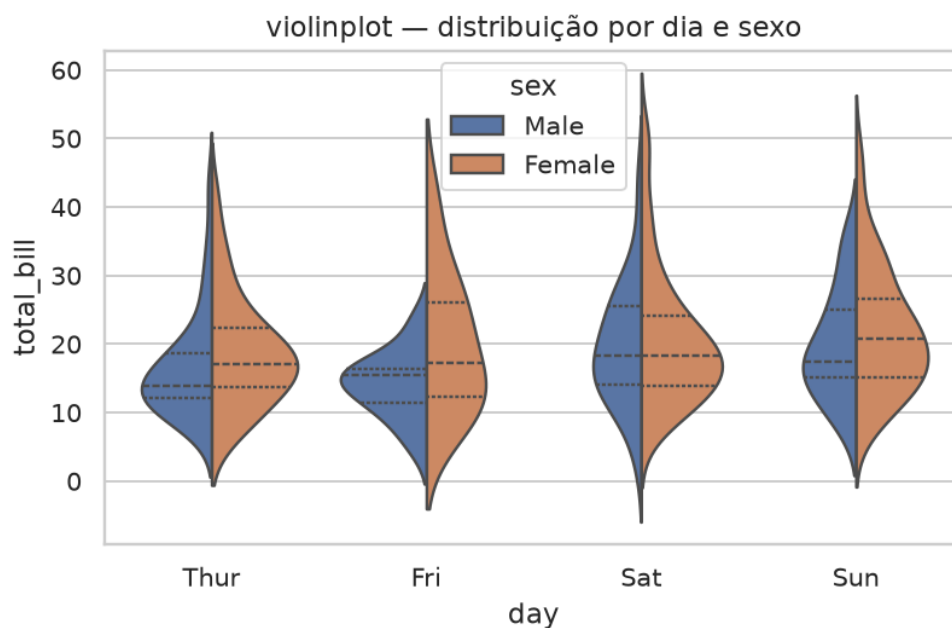
7.1 boxplot e violinplot

```
sns.boxplot(data=tips, x="day", y="total_bill", hue="smoker")
```

```
sns.violinplot(data=tips, x="day", y="total_bill",  
               hue="sex", split=True, inner="quart")
```



boxplot dividido por 'smoker'.

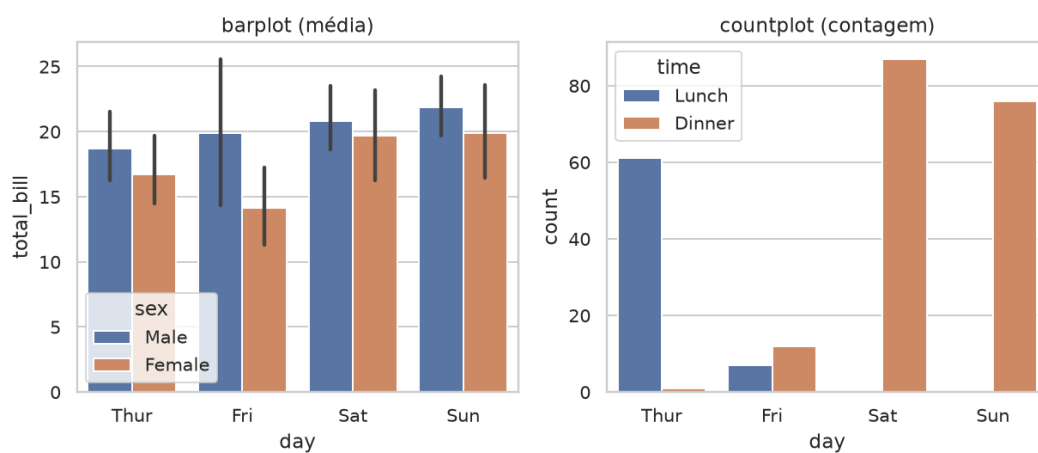


violinplot com split por sexo.

7.2 barplot e countplot

```
sns.barplot(data=tips, x="day", y="total_bill",  
            hue="sex", estimator="mean", errorbar=("ci", 95))
```

```
sns.countplot(data=tips, x="day", hue="time")
```



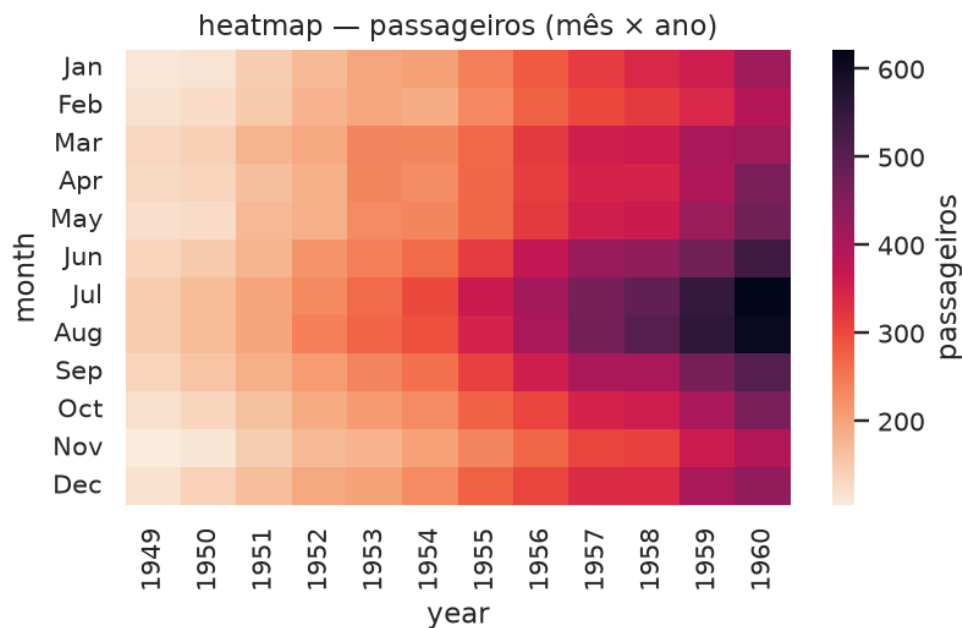
Esquerda: barplot (média). Direita: countplot (contagem).

8. Matrizes — heatmap e clustermap

O **heatmap** exibe uma matriz 2D como cores. Usos comuns: correlações, tabelas dinâmicas, matrizes de confusão.

```
matriz = flights.pivot(index="month", columns="year", values="passengers")
sns.heatmap(matriz, cmap="rocket_r", annot=False,
            cbar_kws={"label": "passageiros"})
```

```
# Correlação entre variáveis numéricas
corr = penguins.select_dtypes("number").corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm",
            center=0, square=True, linewidths=.5)
```



heatmap: passageiros por mês × ano.

O **clustermap** é um heatmap com dendrogramas de agrupamento hierárquico aplicados às linhas e colunas — excelente para descobrir padrões escondidos.

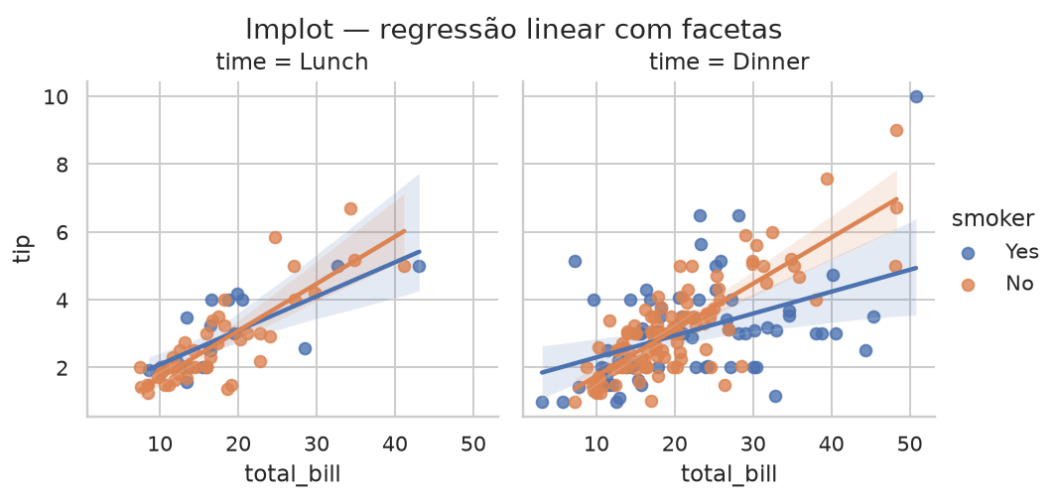
```
sns.clustermap(corr, cmap="vlag", center=0, figsize=(6,6))
```

9. Regressão — regplot e lmlplot

regplot desenha uma dispersão com reta de regressão e intervalo de confiança. **lmlplot** é a versão figure-level que permite facetamento por *hue*, *col*, *row*.

```
sns.regplot(data=tips, x="total_bill", y="tip", order=1) # linear
sns.regplot(data=tips, x="total_bill", y="tip", order=2) # polinomial

sns.lmlplot(data=tips, x="total_bill", y="tip",
            hue="smoker", col="time", height=3.5,
            robust=False, ci=95)
```



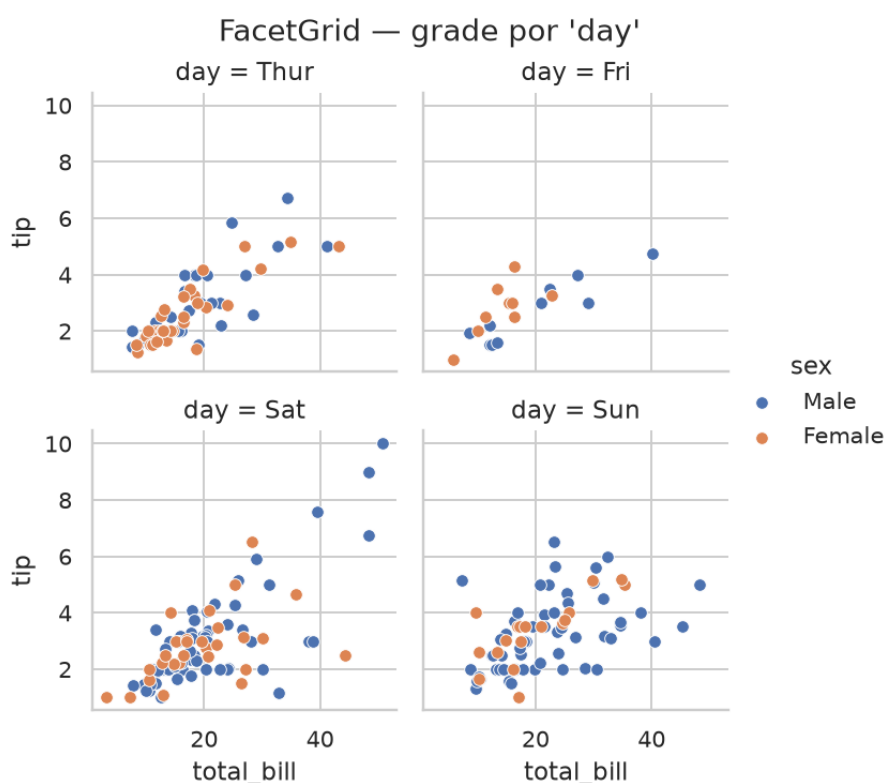
Implot facetado por 'time' e colorido por 'smoker'.

10. Grades de subplots

Grades permitem repetir o mesmo tipo de gráfico para múltiplos subconjuntos de dados (*small multiples*).

10.1 FacetGrid

```
g = sns.FacetGrid(tips, col="day", hue="sex", col_wrap=2, height=2.6)
g.map_dataframe(sns.scatterplot, x="total_bill", y="tip")
g.add_legend()
g.set_axis_labels("Conta (US$)", "Gorjeta (US$)")
g.set_titles("Dia: {col_name}")
```



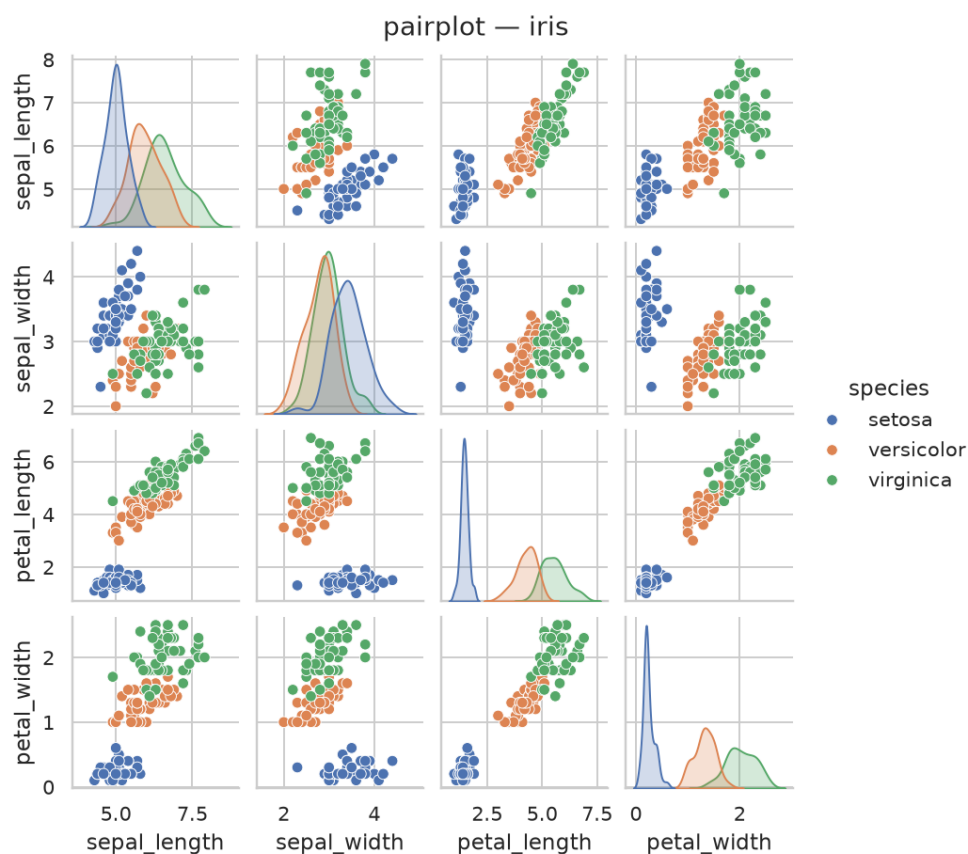
FacetGrid: um subplot por dia da semana.

10.2 PairGrid / pairplot

```
sns.pairplot(iris, hue="species", diag_kind="kde")
```

Versão manual (PairGrid) permite funções diferentes por região

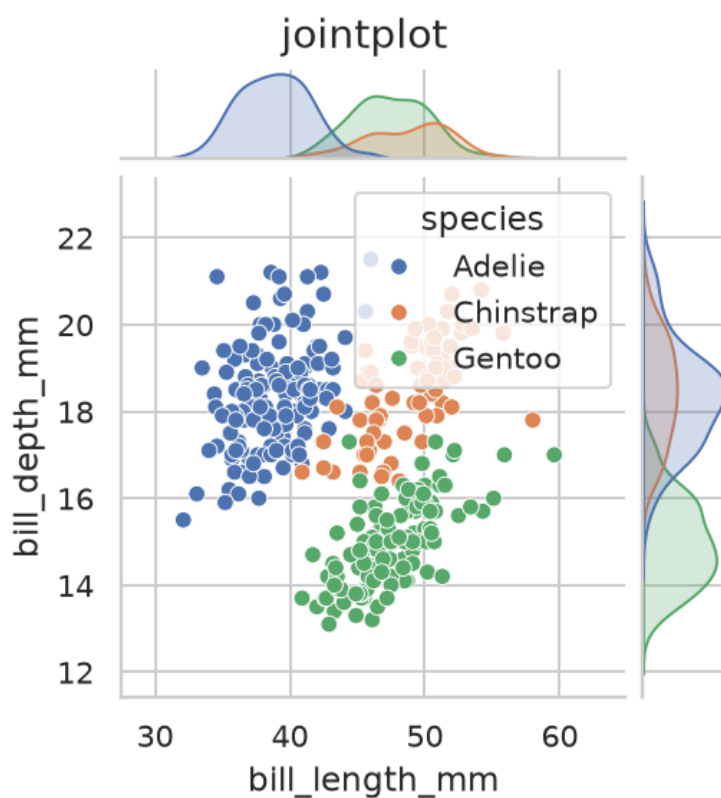
```
g = sns.PairGrid(iris, hue="species")
g.map_upper(sns.scatterplot)
g.map_lower(sns.kdeplot, fill=True)
g.map_diag(sns.histplot)
g.add_legend()
```



pairplot do dataset iris.

10.3 JointGrid / jointplot

```
sns.jointplot(data=penguins,
              x="bill_length_mm", y="bill_depth_mm",
              hue="species", kind="scatter")
# kind: 'scatter' | 'kde' | 'hist' | 'hex' | 'reg' | 'resid'
```



jointplot: dispersão + histogramas marginais.

11. Figure-level vs axes-level

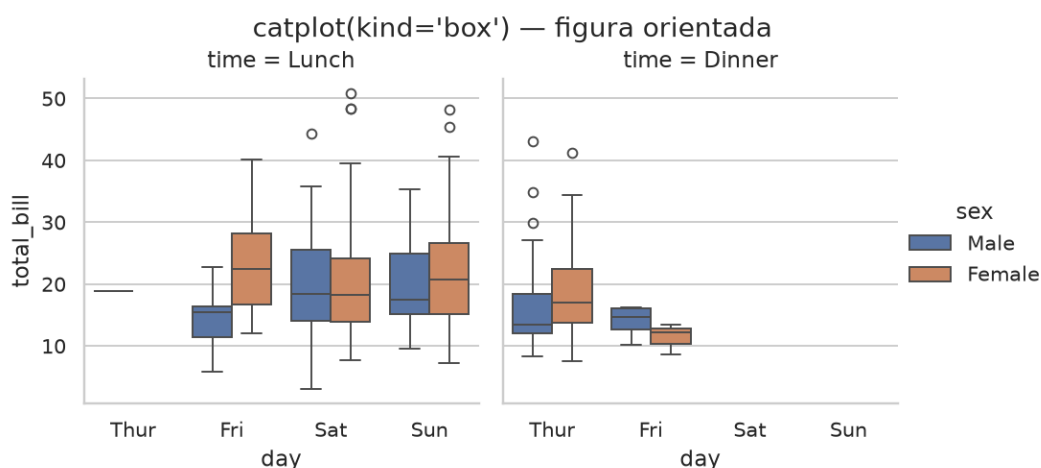
Cada categoria de gráfico tem duas 'famílias' de funções:

Categoria	Axes-level	Figure-level
Relacional	scatterplot, lineplot	relplot
Distribuição	histplot, kdeplot, ecdfplot	displot
Catégorico	boxplot, violinplot, barplot...	catplot
Regressão	regplot, residplot	lmpplot
Matriz	heatmap	(clustermap*)

As funções **axes-level** plotam em um Axes do Matplotlib e aceitam `ax=`. As **figure-level** criam a figura inteira, gerenciam legendas e permitem facetamento com `row=`/`col=`.

```
# axes-level (integra com plt.subplots)
fig, axes = plt.subplots(1, 2, figsize=(10,4))
sns.boxplot(data=tips, x="day", y="total_bill", ax=axes[0])
sns.violinplot(data=tips, x="day", y="total_bill", ax=axes[1])

# figure-level (Seaborn cria a figura)
sns.catplot(data=tips, x="day", y="total_bill",
            hue="sex", col="time", kind="box", height=3.5)
```



12. Customização, títulos e exportação

```
ax = sns.scatterplot(data=tips, x="total_bill", y="tip", hue="time")
ax.set_title("Gorjeta vs Conta", fontsize=14, weight="bold")
ax.set_xlabel("Total da conta (US$)")
ax.set_ylabel("Gorjeta (US$)")
ax.set_xlim(0, 60)
ax.legend(title="Refeição", loc="upper left")
sns.despine() # remove bordas
plt.tight_layout()
plt.savefig("grafico.png", dpi=200, bbox_inches="tight")
plt.savefig("grafico.pdf") # vetor
plt.savefig("grafico.svg")
```

Para figure-level plots, use o objeto retornado:

```
g = sns.catplot(...)
g.set_titles("{col_name}")
g.set_axis_labels("Eixo X", "Eixo Y")
g.figure.suptitle("Título geral", y=1.02)
g.savefig("out.png", dpi=200)
```

13. Integração com Matplotlib e Pandas

Você pode combinar Seaborn e Matplotlib livremente. Também é possível chamar métodos do Pandas (`.plot()`) no mesmo Axes.

```
fig, ax = plt.subplots(figsize=(7,4))
sns.histplot(data=tips, x="total_bill", ax=ax, kde=True, color="#2E5EAA")
ax.axvline(tips["total_bill"].mean(), color="red", ls="--", label="média")
ax.axvline(tips["total_bill"].median(), color="green", ls=":", label="mediana")
ax.legend()
```

Com Pandas, filtre e agregue *antes* de plotar — o Seaborn não é o lugar para transformação pesada de dados.

```
agrupado = (tips
            .groupby(["day", "sex"], as_index=False)["tip"]
            .mean())
sns.barplot(data=agrupado, x="day", y="tip", hue="sex")
```


14. Boas práticas de visualização

1. Comece pela pergunta. Qual insight o gráfico precisa comunicar? Isso define o tipo.

2. Escolha o gráfico certo para o tipo de variável:

Objetivo	Sugestão
Comparar categorias	barplot, boxplot
Ver distribuição	histplot, kdeplot, violinplot
Relação entre 2 numéricas	scatterplot, regplot, jointplot
Tendência ao longo do tempo	lineplot
Correlação entre muitas variáveis	heatmap de corr, pairplot
Detectar outliers	boxplot, stripplot

3. Cuide da cor. Use paletas qualitativas para categorias, sequenciais para valores contínuos e divergentes quando houver um ponto central. Prefira *colorblind* em publicações.

4. Não polua. Remova o que não informa: borda extra, grid excessivo, legendas duplicadas. Chame `sns.despine()`.

5. Rotule tudo. Título claro, eixos com unidade, legenda com título.

6. Salve em alta resolução (dpi=200) ou em formato vetorial (PDF/SVG) para relatórios.

7. Deixe o Seaborn calcular estatística — média, IC, KDE, regressão já vêm embutidos.

15. Referência rápida (cheatsheet)

```
# ---- Setup ----
import seaborn as sns, matplotlib.pyplot as plt, pandas as pd
sns.set_theme(style="whitegrid", palette="colorblind")

# ---- Relacional ----
sns.scatterplot(data=df, x=, y=, hue=, size=, style=)
sns.lineplot(data=df, x=, y=, hue=, errorbar="sd")
sns.relplot(data=df, x=, y=, kind="scatter"|"line", col=, row=)

# ---- Distribuição ----
sns.histplot(data=df, x=, hue=, kde=True, stat="density", bins=30)
sns.kdeplot(data=df, x=, y=, hue=, fill=True)
sns.ecdfplot(data=df, x=, hue=)
sns.displot(data=df, x=, kind="hist"|"kde"|"ecdf", col=)

# ---- Categórico ----
sns.boxplot(data=df, x=, y=, hue=)
sns.violinplot(data=df, x=, y=, hue=, split=True, inner="quart")
sns.barplot(data=df, x=, y=, estimator="mean", errorbar=("ci",95))
sns.countplot(data=df, x=, hue=)
sns.stripplot(data=df, x=, y=, jitter=0.2)
sns.swarmplot(data=df, x=, y=)
sns.pointplot(data=df, x=, y=, hue=)
sns.catplot(data=df, x=, y=, kind="box"|"bar"|"violin"|..., col=)

# ---- Matriz ----
sns.heatmap(matriz, annot=True, fmt=".2f", cmap="coolwarm", center=0)
sns.clustermap(matriz, cmap="vlag", center=0)

# ---- Regressão ----
sns.regplot(data=df, x=, y=, order=1, robust=False)
sns.lmplot(data=df, x=, y=, hue=, col=, row=)
sns.residplot(data=df, x=, y=)

# ---- Grades ----
sns.pairplot(df, hue=, diag_kind="kde")
sns.jointplot(data=df, x=, y=, kind="scatter"|"kde"|"hex"|"reg")
g = sns.FacetGrid(df, col=, row=, hue=)
g.map_dataframe(sns.scatterplot, x=, y=).add_legend()

# ---- Estilo ----
sns.set_style("whitegrid")           # darkgrid | white | dark | ticks
sns.set_context("notebook")         # paper | notebook | talk | poster
sns.set_palette("Set2")
sns.despine(left=True)

# ---- Exportar ----
plt.tight_layout()
plt.savefig("fig.png", dpi=200, bbox_inches="tight")
plt.savefig("fig.pdf")               # vetor
```

Bons plots! ■